

/*

-----French-----

Exercice 1

- 1) Ecrire une fonction Allouer (int taille), qui réserve la place pour un tableau de n entiers. La taille n est lue au clavier.
- 2) Ecrire une fonction saisie(int * T, int taille), qui reçoit le tableau dynamique T et lit au clavier ses éléments.
- 3) Ecrire une fonction Affiche(int * T, int taille), qui affiche les éléments du tableau T.
- 4) Ecrire une fonction Réallouer (int* T, int taille, int N), qui permet de réserver la place pour N nouveaux éléments.
- 5) Ecrire la fonction insereValeurs(int * T, int taille, int ind, int N), insérant les N nombres impaires dans le tableau, à partir de l'indice ind. Il faut gérer la situation, au cas où l'indice ind est en dehors des bornes du tableau.
- 6) Ecrire une fonction tri_ Bulle (int * T, int newsize) qui permet de trier le tableau T par ordre croissant.
- 7) Ecrire la fonction SupprimeDuplications (int * T, int newsize) qui permet de supprimer les éléments en multiples dans le tableau T, et renvoyer la nouvelle taille du tableau. Gérer le problème de l'espace non alloué.
- 8) Définir une fonction compareTableaux (int *T1, int n1, int *T2, int n2, int i), qui vérifie si deux tableaux d'entiers sont identiques, coïncident ou ne coïncident pas.
- 9) Ecrire un programme principal de test.

*/

/*

-----English-----

Exercise 1

- 1) Write a function Allocate(int size), which reserves space for an array of n integers. Size n is read from the keyboard.
 - 2) Write an input function (int * T, int size), which receives the dynamic array T and reads its elements from the keyboard.
 - 3) Write a function Display(int * T, int size), which displays the elements of the array T.
 - 4) Write a function Reallocate (int* T, int size, int N), which allows to reserve space for N new elements.
 - 5) Write the function insertValues(int * T, int size, int ind, int N), inserting the N odd numbers into the array, starting from the index ind. We must manage the situation, in case the index ind is outside the bounds of the array.
 - 6) Write a function tri_ Bulle (int * T, int newsize) which sorts the array T in ascending order.
 - 7) Write the function SuppressDuplications (int * T, int newsiz e) which allows multiple elements to be suppressed in the array T, and returns the new size of the array. Manage the problem of unallocated space.
 - 8) Define a function compareArrays (int *T1, int n1, int *T2, int n2, int i), which checks if two arrays of integers are identical, coincide or do not coincide.
 - 9) Write a main test program. // Functions in use // 1. int *allocate(int size); // 2. void get_elements(int *array, int size); // 3. void display(int *array, int size); // 4. int *reallocate(int *array, int size); // 5. void get_elements_secondary(int *array, int size, int newSize); // 6. int *insert_values(int *array, int size, int data, int pos); // 7. void bubble_sort(int *array, int size); // 8. void remove_duplications(int *array, int size); // 9. void compare_arrays_elements(int *array1, int size1, int *array2, int size2);
- */

```

#include <stdio.h>
#include <stdlib.h>

/* Functions that in use */
int *allocate(int size);
void get_elements(int *array, int size);
void display(int *array, int size);
int *realloc(int *array, int size);
void get_elements_secondary(int *array, int size, int newSize);
int *insert_values(int *array, int size, int data, int pos);
void bubble_sort(int *array, int size);
void remove_duplications(int *array, int size);
void compare_arrays_elements(int *array1, int size1, int *array2, int size2);
int main()
{
    int *array, size, newSize, length, data, pos;
    printf("Enter size of the array: ");
    scanf("%d", &size);
    array = allocate(size);
    get_elements(array, size);
    display(array, size);
    printf("\nEnter size of N new elements: ");
    scanf("%d", &newSize);
    length = size + newSize;
    array = realloc(array, length);
    printf("\nGet secondary elements:_____");
    if (size < length)
        get_elements_secondary(array, size, length);
    else
        printf("We don't have any update for the old array!\n");
    printf("\nAfter adding elements:_____");
    display(array, length);
    printf("\nValue: ");
    scanf("%d", &data);
    printf("\nEnter one of the following position to insert value (%d) at it: \n", data);
    printf("{");
    for (int i = 0; i < length; i++)
        printf("%d, ", i);
    printf("}:\n");
}

```

```

printf("Position = ");
scanf("%d", &pos);
printf("\nAfter inserting the given value:_____ ");
length = length + 1;
array = insert_values(array, length, data, pos);
display(array, length);
printf("\nAfter sorting the elements:_____ ");
bubble_sort(array, length);
display(array, length);
printf("\nDATA after removing duplications:_____ ");
remove_duplications(array, length);
int *array1, size1, *array2, size2;
printf("\n_____PART of ARRAY1_____ \n");
printf("\nSize of array 1 = ");
scanf("%d", &size1);
array1 = allocate(size1);
get_elements(array1, size1);
printf("\nSize of array 2 = ");
scanf("%d", &size2);
array2 = allocate(size2);
get_elements(array2, size2);
printf("\nElements of array 1:_____");
display(array1, size1);
printf("\nElements of array 2:_____");
display(array2, size2);
compare_arrays_elements(array1, size1, array2, size2);
printf("\n");
return 0;

```

```

}

```

```

int *allocate(int size)

```

```

{

```

```

    int *array;
    array = (int *)malloc(size * sizeof(int));
    return array;

```

```

}

```

```

void get_elements(int *array, int size)

```

```

{

```

```

    printf("\n\t***** Get Data *****\n");

```

```

    for (int i = 0; i < size; i++)
    {
        printf("Enter data at %d: ", i);
        scanf("%d", array + i);
    }
}

void display(int *array, int size)
{
    printf("\n\t***** Data That exist in the Array *****\n");
    for (int i = 0; i < size; i++)
        printf("At Index %d | %d\n", i, *(array + i));
}

int *reallocate(int *array, int size)
{
    array = (int *)realloc(array, size * sizeof(int));
    return array;
}

void get_elements_secondary(int *array, int size, int newSize)
{
    printf("\n\t***** Get Data *****\n");
    for (int i = size; i < newSize; i++)
    {
        printf("Enter data at %d: ", i);
        scanf("%d", array + i);
    }
}

int *insert_values(int *array, int size, int data, int pos)
{
    int *array2, i;
    for (i = 0; i < size; i++)
    {
        if (i == pos)
            *(array2 + i) = data;
        if (i < pos)
            *(array2 + i) = *(array + i);
        if (i > pos)
            *(array2 + i) = *(array + i - 1);
    }
}

```

```

        return array2;
    }

void bubble_sort(int *array, int size)
{
    int i, flag, temp;
    do
    {
        flag = 1;
        for (i = 1; i < size; i++)
        {
            if (*(array + i) < *(array + i - 1))
            {
                temp = *(array + i);
                *(array + i) = *(array + i - 1);
                *(array + i - 1) = temp;
                flag = 0;
            }
        }
    } while (flag == 0);
}

void remove_duplications(int *array, int size)
{
    int i, j, k;
    for (i = 0; i < size - 1; i++)
    {
        for (j = i + 1; j < size; j++)
        {
            if (*(array + i) == *(array + j))
            {
                for (k = j; k < size; k++)
                {
                    if (k < j)
                        *(array + k) = *(array + k);
                    if (k >= j)
                        *(array + k) = *(array + k + 1);
                }
                size--;
            }
        }
    }
}

```

```

        else
            j++;
    }
}
display(array, size);
}

void intersection_elements(int *array1, int size1, int *array2, int size2);
void union_elements(int *array1, int size1, int *array2, int size2);
int *merge(int *array1, int *array2, int size1, int size2);
void compare_arrays_elements(int *array1, int size1, int *array2, int size2)
{
    printf("\nAfter comparing:____INTERSECTIONS_____\n");
    intersection_elements(array1, size1, array2, size2);
    printf("\n\nAfter comparing:____UNIONS_____\n");
    union_elements(array1, size1, array2, size2);
}

void intersection_elements(int *array1, int size1, int *array2, int size2)
{
    for (int i = 0; i < size1; i++)
    {
        for (int j = 0; j < size2; j++)
        {
            if ((*array1 + i) == (*array2 + j))
                printf("%d\t", *(array1 + i));
        }
    }
}

int *merge(int *array1, int *array2, int size1, int size2)
{
    int *array, size, i, j = 0;
    size = size1 + size2;
    array = allocate(size);
    for (i = 0; i < size1; i++)
    {
        *(array + j) = *(array1 + i);
        j++;
    }
    for (i = 0; i < size2; i++)

```

```
{  
    *(array + j) = *(array2 + i);  
    j++;  
}  
return array;
```

```
}
```

```
void union_elements(int *array1, int size1, int *array2, int size2)
```

```
{
```

```
    int *array, size;
```

```
    size = size1 + size2;
```

```
    array = allocate(size);
```

```
    array = merge(array1, array2, size1, size2);
```

```
    remove_duplications(array, size);
```

```
}
```

/*

Edit by: MOHAMMED ELIDRISSI LAOUKILI
#idriCoding | #idriLogic | #idri
2021/2022

لا اله الا الله

*/